

NLP Project: Sentiment Classification

Project 1.4 - Group 11

Fabian Beez

fabian.beez@campus.tu-berlin.de

Technische Universität Berlin

Berlin, Germany

Muhammed Yavuz

m.yavuz@campus.tu-berlin.de

Technische Universität Berlin

Berlin, Germany

ABSTRACT

In this work, we analyze and classify sentiment in a labeled Twitter dataset containing positive, negative, and neutral tweets. We begin with an exploratory data analysis, revealing sentiment-related patterns in lexical choices, emoticon usage, temporal trends, part-of-speech distributions, and more. A configurable preprocessing pipeline with 17 modular steps is implemented to flexibly clean and transform raw tweet texts, enabling controlled experimentation. For sentiment classification, we train and evaluate multiple models - a Naïve Bayes classifier, two feed-forward neural networks, and a recurrent neural network (RNN) - on both the original three-class setup and a binary classification setting (positive vs. negative), enabling focused sentiment detection. We assess various vectorization techniques and preprocessing configurations, finding that including n-grams can boost performance, while excessive preprocessing may reduce effectiveness. Error analysis highlights label noise and semantic ambiguity as key challenges. Additionally, we compute pointwise mutual information (PMI) scores to explore word similarities within sentiment classes. Our work provides insights into both the linguistic structure of social media text and practical considerations for building robust sentiment classification pipelines.

1 INTRODUCTION

Sentiment analysis of social media texts like tweets has become an important task in natural language processing due to the vast amount of user-generated content reflecting public opinion. Effective text classification in this context is challenging because of informal language, noise, and limited context. Therefore, improving preprocessing and feature extraction methods plays a crucial role in enhancing the accuracy of sentiment classifiers.

To address these challenges, this report begins by introducing the dataset in Chapter 2, followed by a detailed overview of applicable preprocessing techniques in Chapter 3. Chapter 4 presents the results of text classification experiments, and Chapter 5 investigates PMI-based word similarity. Finally, Chapter 6 summarizes the findings and discusses their meaning.

2 DATASET

To gain insights from the Twitter dataset, we conducted a series of exploratory analyses. The dataset contains tweets labeled by sentiment (positive, neutral, or negative), along with several additional statistical columns like the time of the tweet. We extracted key insights through various analysis methods, which are presented in the following subsection.

2.1 Insights

Dataset Balance. We analyzed the sentiment distribution to assess potential class imbalance. The dataset contains 11118 neutral, 8582 positive, and 7781 negative tweets, revealing a moderate imbalance, with neutral tweets being the majority.

Frequent Words by Sentiment. We analyzed the most frequent words per sentiment by removing stopwords and computing word frequencies. To visualize the results, we additionally generated word clouds for each sentiment category without stopword filtering. Common words across all sentiments include “im” and “now”, while sentiment-specific words also emerge:

- **Positive:** love, thank, day, good, mothers
- **Negative:** miss, work, sad, sorry
- **Neutral:** today, u, lol, day, work, one, go

The full results can be viewed in `1-insights.ipynb`.

Text Lengths. To analyze tweet length distributions, we used box plots and violin plots for deeper insight. On average, neutral tweets are shorter than emotional ones, with longer tweets occurring mostly in the positive and negative sentiment classes.

TF-IDF Bigrams. To identify characteristic phrases for each sentiment category, we computed and ranked bigrams based on their Term Frequency–Inverse Document Frequency (TF-IDF) scores. This highlights two-word combinations that are particularly distinctive within each sentiment. Many of the top-ranked bigrams were URLs, such as “twitpic com” and “http twitpic”, suggesting frequent link sharing. Notable non-URL bigrams include:

- **Neutral:** don know, just got
- **Negative:** feel like, sorry hear, just got, don know, don like
- **Positive:** mother day, happy mother, mothers day, happy mothers, good morning, star wars, happy birthday, wars day

The frequent appearance of phrases such as “mother day” and “star wars” suggests the dataset may have been collected within a short time span in early May, coinciding with Star Wars Day (May 4th) and Mother’s Day (the second Sunday of May).

TF-IDF Trigrams. Extending this analysis to trigrams further supported our results. The results confirm earlier insights, as many high-ranking three-word phrases reference the same context. The top trigrams for each class are:

- **Neutral:** just got home, work work work, happy mothers day
- **Positive:** happy mother day, happy mothers day, star wars day
- **Negative:** don feel good, dont feel good, just got home

Interestingly, the trigram “http yfrog com” (a former image hosting service for Twitter) only appears in the negative class.

Lexical Richness per Sentiment. We analyzed the number of unique words, excluding stopwords and punctuation, used within each sentiment category. Neutral tweets exhibit the most diverse vocabulary, while negative tweets use the fewest unique words. This pattern is likely influenced by the dataset imbalance, as the neutral class contains more tweets than the others. Additionally, the corpus-level lexical diversity indicates that positive tweets tend to be slightly more repetitive, whereas neutral and negative tweets show similar levels of lexical variety.

Part-of-Speech Distribution. The POS tag distribution is relatively consistent across all sentiment classes. Most POS tags occur most frequently in the neutral class, which aligns with its larger number of samples. This indicates a similar grammatical structure across sentiments despite lexical differences.

Topic Modeling. We applied Latent Dirichlet Allocation (LDA) separately to each sentiment category to uncover underlying themes. For each class, three topics were extracted, with the top 10 most relevant words visualized. The topics reveal meaningful patterns:

- **Neutral:** Tweets were commonly referencing daily routines and mundane activities with words such as *work*, *home*, and *morning*.
- **Negative:** Expressing emotional distress and complaints, highlighted by terms like *sad*, *miss*, *sorry*, *hate*, and *work*.
- **Positive:** Topics express joy and appreciation, frequently using words such as *love*, *good*, *weekend*, and *mothers day*.

These results align with earlier analyses and reinforce the impression that many tweets are related to temporal events like Mother’s Day, suggesting the dataset may have been collected within a short time span.

Semantically Relevant Stopwords. We examined common stopwords and their potential semantic relevance across different sentiment classes. While such words are typically removed during preprocessing, some can carry important contextual information. One example for this is the word “but” - it appears most frequently in neutral tweets (around 10%), followed by negative ones (9%), and is least used in positive tweets (6%). This suggests that certain stopwords may reflect sentiment-specific discourse structures and should not always be discarded blindly.

Emoticons. We examined the distribution of emoticons across sentiment classes by scanning the text for common emoticons. Initially, the emoticon “:\” was included in the analysis, but it often appeared in URLs, especially in neutral tweets, which skewed the results. After excluding this symbol, we found that the most frequently used emoticons are “<3”, “;)”, and “:3”. Overall, emoticons appear predominantly in positive or neutral tweets and are rarely used in negative ones, suggesting their association with more favorable or neutral emotional expression.

Demographic and Temporal Analysis. We analyzed the distribution of sentiments based on demographic and temporal data, including the time of the tweet, the author’s age group, and the country of origin. Sentiments are relatively evenly distributed across age

groups and time of day, suggesting that neither factor has a significant influence on sentiment. For country-level differences, we created an interactive chart. While some variation exists between countries, these may be due to statistical noise rather than meaningful patterns.

2.2 Summary

In this chapter, we conducted exploratory analyses revealing a moderate imbalance with mostly neutral tweets. Distinct vocabulary and topics correspond to the expressed emotions and highlight temporal events such as Mother’s Day and Star Wars Day, indicating the dataset was likely collected within a short time frame. Grammatical patterns were consistent across sentiments, while emoticons and some stopwords showed sentiment-specific usage. Demographic and temporal factors had minimal impact on sentiment distribution. These findings help characterize the dataset for our later classification.

3 PREPROCESSING

To prepare the raw Twitter data for our classification task, we defined a single configurable preprocessing function. This function enables flexible activation of any subset of 17 different preprocessing steps, allowing us to tailor the cleaning to our needs. The implementation uses the *NLTK* library for linguistic tasks and a spellchecker library for spelling correction.

Tokenization is integrated as part of the pipeline, occurring after initial text cleaning. This allows subsequent operations such as stemming, lemmatization, stopword removal, and elongation reduction to be performed on tokenized words before recombining the cleaned tokens into a string.

The available preprocessing steps include:

- **Lowercase:** Convert all characters in the text to lowercase.
- **Strip Multispace:** Remove multiple consecutive whitespaces.
- **Expand Contractions:** Replace common contractions (e.g., “don’t” → “do not”).
- **Remove URLs:** Eliminate links from the text.
- **Remove Mentions:** Remove usernames (e.g., “@user”).
- **Emoticon Normalization:** Replace common emoticons with descriptive tokens (e.g., “:)” → “SMILE”).
- **Detect Censored Words:** Replace sequences of asterisks (e.g., “****”) with the placeholder “CENSORED”.
- **Remove Punctuation:** Remove all punctuation marks from the text.
- **Preserve Ellipsis:** Preserve ellipses (“...”) while removing other punctuation.
- **Remove Numbers:** Remove all numeric digits from the text.
- **Convert Numbers:** Convert numbers into their word form (e.g., “12” → “twelve”).
- **Remove Non-ASCII Characters:** Eliminate non-standard characters such as emojis and accented letters.
- **Spelling Correction:** Correct misspelled words using a spellchecker.
- **Reduce Elongation:** Normalize repeated characters (e.g., “soooo” → “soo”).

- **Remove Stopwords:** Remove common English stopwords (e.g., “the”, “is”).
- **Stemming:** Apply Porter stemming to reduce words to their root forms (e.g., “running” → “run”).
- **Lemmatization:** Perform lemmatization using WordNet and part-of-speech tags to reduce words to their base form.

This modular design enables easy adaptation of preprocessing to different experimental settings by simply toggling the corresponding flags when calling the function.

4 TEXT CLASSIFICATION

To prepare the dataset for sentiment classification, we applied over-sampling using a custom `get_balanced_dataset` function. This function balances the class distribution by resampling each sentiment category to match the size of the largest class. After balancing, we performed a stratified train-test split to preserve the class distribution across both sets, ensuring a fair evaluation. All random seeds were fixed to 42 for reproducibility.

We then implemented and evaluated multiple classifiers across various vectorization and preprocessing configurations to systematically assess their performance. Finally, we conducted error analysis to identify common misclassification patterns and better understand the limitations of each model.

4.1 Naïve Bayes

We first implemented a Naïve Bayes classifier as a fast, scalable probabilistic baseline. Its simplicity makes it ideal for high-dimensional text and a useful reference for evaluating vectorization and preprocessing choices.

Vectorizers. To assess the influence of different vectorization methods on Naïve Bayes performance, we compared Bag of Words (BoW), BoW with bigrams, TF-IDF, a Binary vectorizer, and a Hashing Vectorizer with 10000 features. Results are shown in Table 2.

Surprisingly, BoW, TF-IDF, and binary vectorization performed nearly identically, despite encoding different levels of information. This may be attributed to the relatively small size and simplicity of the dataset, or to the Naïve Bayes model’s insensitivity to term weighting schemes. In contrast, BoW with bigrams showed improved performance, as expected, due to its ability to capture short semantic structures such as “not good”. The Hashing Vectorizer underperformed, which aligns with expectations due to hash collisions.

Preprocessing. Since BoW with bigrams achieved the best performance in previous evaluations, we selected this vectorizer to examine the effect of various preprocessing strategies. Surprisingly, the impact of individual preprocessing steps was minimal. Even when all preprocessing options were applied together, the overall classification performance dropped slightly by about 1%.

This indicates that the Naïve Bayes model is relatively robust to textual noise, likely due to the dataset’s small size and the dominance of strong sentiment-bearing keywords such as “happy”, “Mother’s Day”, or “hate” that drive classification outcomes regardless of preprocessing. Detailed performance metrics for the different preprocessing configurations are presented in Table 1.

Table 1: Impact of individual preprocessing options on model accuracy (BoW + bigrams; baseline NB: 0.8357, NN: 0.8766)

Preprocessing Option	Naïve Bayes	Small NN
Lowercase	-0.0010	+0.0006
Expand contractions	-0.0010	+0.0000
Remove URLs	-0.0025	+0.0016
Emoticon normalization	-0.0003	-0.0004
Detect censored words	+0.0007	+0.0021
Remove mentions	-0.0007	+0.0000
Remove punctuation	-0.0024	-0.0046
Preserve ellipsis	-0.0010	+0.0010
Remove numbers	-0.0016	-0.0007
Convert numbers	-0.0012	-0.0006
Remove non-ASCII chars	-0.0013	-0.0006
Reduce elongation	+0.0000	+0.0012
Remove stopwords	-0.0037	-0.0043
Stemming	+0.0016	+0.0024
Lemmatization	+0.0013	+0.0024
Spelling correction	-0.0013	-0.0022
Strip multi-space	-0.0010	-0.0007

Error Analysis. The confusion matrix in Figure 1 of the Naïve Bayes classifier with BoW and bigrams reveals a balanced distribution of classification numbers across all sentiment classes. This balance explains the consistent precision, recall, and F1-score values, indicating that the model does not exhibit bias toward any particular class. However, the accuracy on positive and negative samples is much higher compared to neutral samples, probably because they are misclassified more easily. Assigning them a higher weight when balancing the dataset might resolve this.

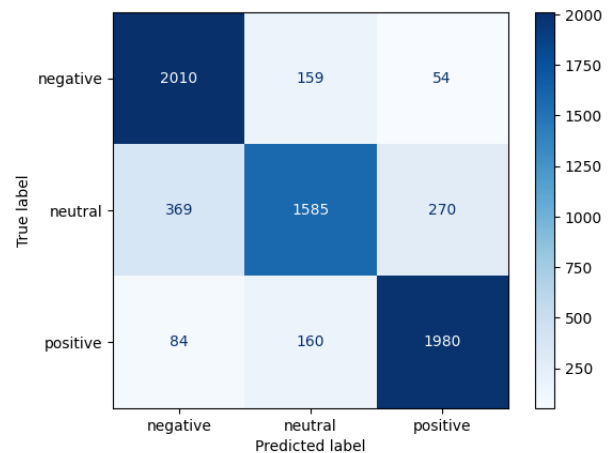


Figure 1: Confusion matrix for Naïve Bayes model using BoW with bigrams

To better understand model behavior, we manually reviewed misclassified samples sorted by prediction confidence. High-confidence misclassifications were often attributable to labeling errors, e. g.

“happy mothers day to all you mums out there” was labeled as negative in the dataset. Medium-confidence errors typically involved tweets that were contextually ambiguous or subjective, being hard to classify, such as “awwwwww...I know that feeling!!”, where emotional tone depends heavily on interpretation. Low-confidence predictions corresponded to inputs with minimal semantic content or unclear intent, including examples like “Everything”, “to doente”, and “tweettttt”, which lack sufficient cues for reliable classification.

4.2 Feed-Forward Networks

As a second classification architecture, we investigated feed-forward neural networks by training two models of different capacities. The smaller network consists of a single hidden layer with 128 units followed by a Rectified Linear Unit (ReLU) activation and a final output layer. The larger model includes two hidden layers with 256 and 128 units, respectively, each followed by ReLU activations and dropout for regularization. As before, we examined the impact of various vectorization techniques and preprocessing setups, and conducted error analysis to better understand the models’ performance and limitations.

Vectorizers. Unlike in the Naïve Bayes setup, both of our neural networks exhibited moderate performance differences among the simple unigram-based vectorization methods. This suggests that the earlier similarity between Bag-of-Words, TF-IDF, and binary vectorization likely stemmed from the insensitivity of the Naïve Bayes classifier to vector weights, rather than the inherent characteristics of the dataset or features.

The larger network, trained with increased capacity and over 15 instead of 10 epochs, improved accuracy by 0-3% depending on the vectorization method, at the cost of significantly longer training time. An interesting insight is that the performance advantage of bigram vectorization did not improve with the larger model, suggesting that the more expressive model may already capture some of the contextual dependencies that bigrams explicitly introduce, like shown in Table 2.

Preprocessing. As with the Naïve Bayes model, we evaluated the effect of various preprocessing strategies on both feed-forward networks using bigram-based Bag of Words vectorization. Applying all preprocessing steps led to a slight performance decrease of about 1%, suggesting that the models are robust to textual noise and may depend on surface-level patterns that preprocessing may obscure. Interestingly, the larger network showed marginally higher sensitivity to preprocessing, though this may be due to statistical variation, shown in Table 1.

Error Analysis. Since the smaller feed-forward network with BoW bigrams performed exactly the same as the large network with BoW bigrams, we chose to analyze the output of the smaller model only. Similar to the Naïve Bayes classifier, the confusion matrix in Figure 2 also demonstrates a balanced pattern of classification numbers across sentiment classes. This symmetry aligns with the high and consistent precision, recall, and F1-scores observed during evaluation. An interesting insight is that almost all performance gains in comparison to the Naïve Bayes classifier occur on the neutral class, where accuracy rises by almost 20%, making the

network perform much more similarly on each class than the earlier classifier.

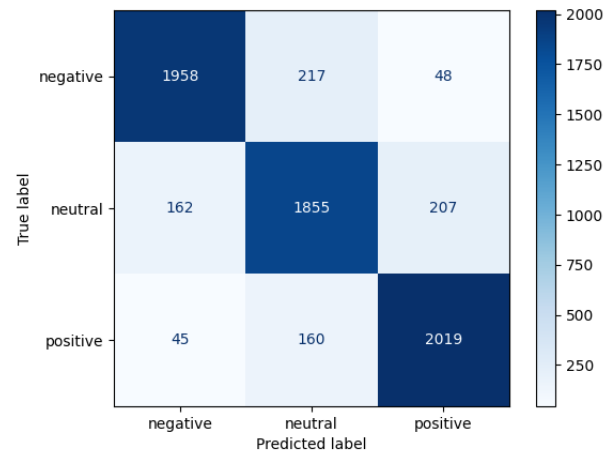


Figure 2: Confusion matrix for small Feed-Forward NN using BoW with bigrams

4.3 Recurrent Neural Networks

Additional to the previous classifiers, we implemented a hybrid model combining a pretrained BERT encoder with either an LSTM or a GRU to capture sequential patterns. BERT’s weights are frozen, allowing the recurrent layers and classifier to specialize in sentiment prediction. This approach leverages both contextual embeddings and sequence modeling for improved classification.

Hyperparameter Optimization. Due to computational constraints, extensive hyperparameter optimization was not feasible. Nevertheless, we conducted a limited search and identified the best-performing configuration within the tested space. The optimal parameters were a hidden dimension of 1024, dropout rate of 0.3, learning rate of 1e-3, LSTM as the recurrent unit, batch size of 32, and training for 15 epochs.

Error Analysis. The RNN further improves classification performance compared to the previous networks, however, at the cost of a much longer training duration. The performance metrics from the RNN are shown in Table 2.

Interestingly, the confusion matrix in Figure 3 shows that the accuracy mainly (and only slightly) increases for positive and negative samples, indicating that the RNN is particularly effective at reducing misclassifications of these more distinct classes. Wrongly classified samples look roughly the same as the ones of the feed-forward network.

4.4 Binary Classification Setting

In the binary classification setup, we excluded neutral tweets and trained the models exclusively on positive and negative examples. This simplification eased the task and led to a noticeable improvement in performance for all models. The RNN achieved an accuracy and F1-score of 0.97, while the feed-forward network and the Naïve Bayes classifier closely followed with 0.96 and 0.95, respectively.

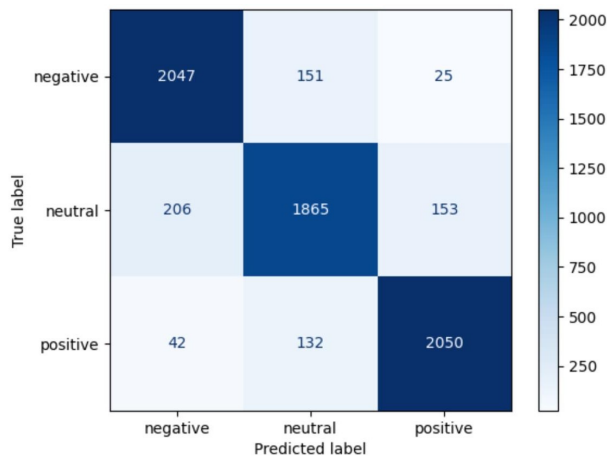


Figure 3: Confusion matrix for BERT RNN

The significant improvement is likely due to the clearer distinction between positive and negative sentiment, as the neutral class in the multiclass setup introduces additional ambiguity. The confusion matrices of each network are visualized in Figures 5, 4, and 6.

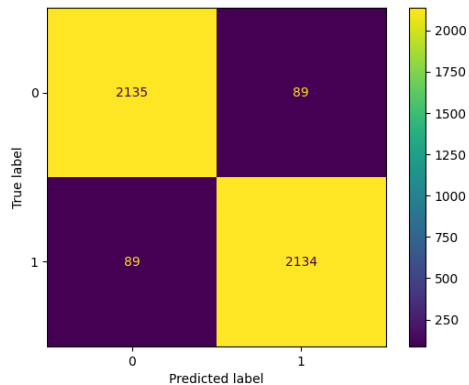


Figure 4: Confusion matrix for binary Naïve Bayes using BoW with bigrams

4.5 Comparison of Classifiers

Overall Performance. The feed-forward neural network outperforms the Naïve Bayes classifier across all vectorization methods. For instance, the Naïve Bayes model using BoW with bigrams achieved a highest accuracy of **0.8357**, while the best neural network (NN) reached up to **0.8745** with the same vectorizer. This trend holds for macro F1 as well, with the NN scoring up to **0.8746** compared to **0.8335** for Naïve Bayes. These results suggest that the NN better captures non-linear relationships in the feature space, especially with richer vectorizations like n-grams. Moreover, the RNN outperformed both models, achieving an accuracy of **0.8937** and a macro F1-score of **0.8934**, highlighting the advantage of contextualized embeddings and sequential modeling.

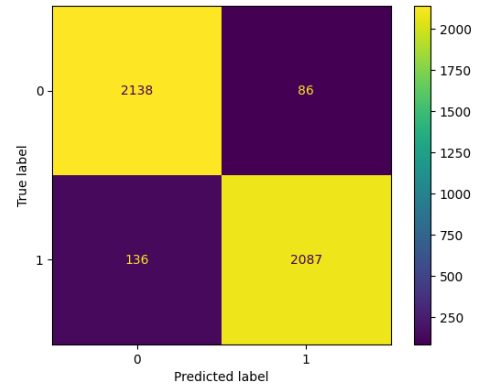


Figure 5: Confusion matrix for binary Feed-Forward NN using BoW with bigrams

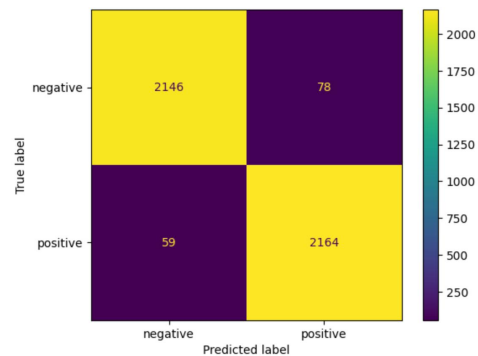


Figure 6: Confusion matrix for binary BERT RNN

Vectorizer Sensitivity. Naïve Bayes showed limited sensitivity to different unigram-based vectorizers like TF-IDF, BoW, and Binary Vectorization. However, the NN responded more distinctly to vectorization changes, showing greater variance in results. Notably, the advantage of bigram BoW diminished in the larger NN, likely because the increased model capacity enables the network to learn complex nonlinear combinations of unigram features, effectively capturing some patterns that n-grams explicitly represent, thus reducing their relative benefit.

Error Patterns. The confusion matrices for all models show similar patterns: most errors occur between neighboring sentiment classes, such as neutral tweets misclassified as positive or negative. Neural networks notably improve performance on the neutral class compared to a Naïve Bayes classifier, while the RNN further reduces misclassification in the positive and negative classes.

Preprocessing. The impact of preprocessing techniques on both Naïve Bayes and neural network models was generally minimal. Individually, each preprocessing option did not result in significant performance changes. Furthermore, when all pre-processing steps were combined, a slight decrease in accuracy of approximately 1%

Table 2: Classification Metrics

Model	Vectorizer	Accuracy	Macro F1	F1 (Neg)	F1 (Neu)	F1 (Pos)
Naïve Bayes	CountVectorizer (BoW)	0.7753	0.7722	0.8038	0.6884	0.8244
	TF-IDF	0.7771	0.7743	0.8047	0.6925	0.8258
	Binary Vectorizer	0.7783	0.7752	0.8061	0.6955	0.8241
	BoW with Bigrams	0.8357	0.8335	0.8579	0.7679	0.8746
	HashingVectorizer	0.7108	0.7091	0.7403	0.6193	0.7676
Small Feed-Forward NN	CountVectorizer (BoW)	0.8645	0.8643	0.8774	0.8241	0.8916
	TF-IDF	0.8480	0.8476	0.8682	0.8016	0.8731
	Binary Vectorizer	0.8667	0.8665	0.8811	0.8269	0.8914
	BoW with Bigrams	0.8730	0.8731	0.8913	0.8315	0.8964
	HashingVectorizer	0.8267	0.8262	0.8472	0.7744	0.8570
Large Feed-Forward NN	CountVectorizer (BoW)	0.8745	0.8746	0.8884	0.8328	0.9026
	TF-IDF	0.8637	0.8630	0.8807	0.8176	0.8905
	Binary Vectorizer	0.8744	0.8742	0.8854	0.8347	0.9024
	BoW with Bigrams	0.8721	0.8725	0.8856	0.8317	0.9003
	HashingVectorizer	0.8541	0.8539	0.8700	0.8080	0.8837
BERT RNN	—	0.8937	0.8934	0.9062	0.8532	0.9209

was actually observed for both models. This suggests that our pre-processing pipeline might obscure important information and that our classification might be guided by strong signal words.

5 PMI-BASED WORD SIMILARITY

To explore semantic similarity in the tweet corpus, we computed the **Pointwise Mutual Information (PMI)** between word pairs. PMI measures how much more often two words co-occur than expected by chance:

$$\text{PMI}(w_1, w_2) = \log \frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)}$$

where $P(w_1, w_2)$ is the joint probability of co-occurrence, and $P(w_1)$, $P(w_2)$ are the individual probabilities.

5.1 Implementation

We preprocessed the tweets by tokenizing and removing stopwords, punctuation, numbers, usernames and special characters. Word co-occurrences were counted using a sliding window of size 1 to build a word-word co-occurrence matrix. From this, we computed PMI scores and constructed word vectors from matrix rows.

To assess semantic similarity, we computed cosine similarities between word vectors. Ten randomly sampled, frequent words (occurring at least ten times) were selected, and their five most similar neighbors were identified.

We implemented two variants of this process: one with only stopword removal and another with full preprocessing, which we inspect in the following analysis. We implemented the additional preprocessing options since many of the top similar words included usernames or irrelevant tokens such as “thru”, “xxxxxxx”, and “sasha”, which are neither affective nor contextually meaningful.

5.2 Results

The resulting word similarities reveal contextual proximity. This is a list of all ten randomly selected words and their pointwise mutual words:

- (1) **age**: deathstar, picnic, gratification, programming, rebootiness
- (2) **called**: erocka, jah, autistics, cingular, vw
- (3) **sign**: digestif, someplace, neighbourhood, wolfram, elevated
- (4) **weird**: tainted, alo, insomnia, sarcy, kryptik
- (5) **original**: omedetou, enlightening, bets, titanic, etonline
- (6) **hear**: kerbear, adrii, morley, inshaa, timing
- (7) **exams**: midterms, lays, assignments, yearly, coomee
- (8) **lives**: lads, boingo, chilliness, shrt, downed
- (9) **night**: nite, morning, day, friday, saturday
- (10) **saved**: wathing, dissection, airy, burp, transferred

For instance, words such as “night”, “morning” and “day” appeared close to other affective or event-related terms, validating PMI as a useful proxy for semantic similarity in short, informal text like tweets.

5.3 Summary

Similar to our earlier TF-IDF weight analysis, PMI-based vectorization enabled us to uncover word associations from the dataset without relying on pre-trained embeddings. However, our findings confirm that the quality of PMI results is highly sensitive to input cleanliness. Applying targeted preprocessing steps significantly improved the interpretability of word “neighborhoods” by reducing noise and removing structurally irrelevant tokens. This demonstrates that even unsupervised techniques like PMI benefit greatly from thoughtful data preparation.

6 CONCLUSION

In this project, we conducted a comprehensive study on tweet-based sentiment classification. We began by analyzing the dataset's structure and temporal aspects, noting that the data was likely collected within a short period, around early May. A configurable preprocessing pipeline was implemented, though experiments showed that individual preprocessing steps had little impact on performance, and full preprocessing even slightly reduced accuracy.

We evaluated multiple models: a Naïve Bayes baseline, two feed-forward neural networks, and an RNN incorporating BERT embeddings. Naïve Bayes consistently underperformed, particularly on

the neutral class, while neural networks achieved more balanced results. The RNN further improved performance, mainly on positive and negative samples.

Error analysis revealed that many misclassifications stemmed from ambiguous or inconsistently labeled tweets. Overall, while simpler models offered competitive baselines, neural networks (especially those using contextual embeddings) demonstrated superior generalization, underscoring their effectiveness for noisy, user-generated content like tweets.